

OSSP-PRACTICAL-WEEK2

Task-1: Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Source code:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("source.txt", O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    printf("Source file opened successfully.\n");
    printf("Destination file opened successfully.\n");

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to destination file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead < 0) {
        perror("Error reading source file");
    } else {
        printf("File contents copied successfully.\n");
    }

    close(source);
    close(destination);

    printf("Files closed successfully.\n");

    return 0;
}
```

Output:

```
root@LAPTOP-1C0JKKG6:~# nano week2pt1.c
root@LAPTOP-1C0JKKG6:~# gcc week2pt1.c -o week2pt1
root@LAPTOP-1C0JKKG6:~# echo "This is a sample file for system call demonstration." > source.txt
root@LAPTOP-1C0JKKG6:~# cat source.txt
This is a sample file for system call demonstration.
root@LAPTOP-1C0JKKG6:~# ./week2pt1
Source file opened successfully.
Destination file opened successfully.
File contents copied successfully.
Files closed successfully.
root@LAPTOP-1C0JKKG6:~#
```

Observation: I learned how Linux system calls such as `open()`, `read()`, `write()`, and `close()` allow a C program to interact with files. I also understood that these operations involve communication between user space and the Linux kernel, where the kernel provides the required file-handling services.

Task2: Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

Output:

```
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_CTYPE", O_RDONLY|O_
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_CTYPE", O_RDONLY|O_C
fstat(3, {st_mode=S_IFREG|0644, st_size=360460, ...}) = 0
mmap(NULL, 360460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c96837a700
close(3) = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...})
openat(AT_FDCWD, "sample.txt", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=41, ...}) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMO
read(3, "Hello, this is a sample file for...", 131072) = 41
write(1, "Hello, this is a sample file for...", 41Hello, this is
) = 41
read(3, "", 131072) = 0
munmap(0x7c9683785000, 139264) = 0
close(3) = 0
close(1) = 0
close(2) = 0
exit_group(0) = ?
+++ exited with 0 +++
root@laptop-1CQJKKG6:~#
```

Observation: I learned how the strace utility can be used to observe the system calls made by a Linux command. I understood how commands such as cat interact with the Linux kernel through system calls to open, read, display, and close a file.